

EN3160 – Assignment 2
Fitting and Alignment

Perera W.A.L.S.
Index No: 230487D

1. Question 1: Sunflower Detection Using Scale-Space Extrema

Range of σ: 15 values from 5 to 10 (step ≈ 0.357). Maxima are searched only at the inner scales, so the largest detectable circles have $\sigma \approx 9.64$, i.e. radius $r = \sqrt{2}\,\sigma \approx 13.6$ px (quarter-resolution image).

```
im = cv.imread('Images/the_berry_farms_sunflower_field.jpeg', cv.IMREAD_REDUCED_COLOR_4)
gray = cv.cvtColor(im, cv.COLOR_BGR2GRAY).astype(np.float32) / 255.0
sigmas = np.linspace(5.0, 10.0, num=15)

scale_space = []
for sigma in sigmas:
    ksize = int(2 * np.ceil(3 * sigma) + 1)
    blur = cv.GaussianBlur(gray, (ksize, ksize), sigma)
    lap = cv.Laplacian(blur, cv.CV_32F)
    scale_space.append((sigma ** 2) * np.abs(lap))      # scale-normalised LoG
scale_space = np.array(scale_space)

detections, threshold = [], 0.02
for s in range(1, len(sigmas) - 1):
    for y in range(1, gray.shape[0] - 1):
        for x in range(1, gray.shape[1] - 1):
            value = scale_space[s, y, x]
            if value < threshold:
                continue
            neighborhood = scale_space[s - 1:s + 2, y - 1:y + 2, x - 1:x + 2]
            if value == np.max(neighborhood):           # 3x3x3 local maximum
                detections.append((y, x, s))

output = im.copy()
for y, x, s in detections:
    r = int(np.sqrt(2) * sigmas[s])                     # radius r = sqrt(2) * sigma
    cv.circle(output, (x, y), r, (0, 255, 0), 1)
```



Figure 1: Detected blobs (green circles of radius $\sqrt{2}\,\sigma$).

2. Question 2: Line and Circle Fitting Using RANSAC

```
def ransac_line(pts, max_iters=1000, thresh=1.0):
```

```
    best_inliers, best_sample, best_params = [], None, None
    for _ in range(max_iters):
        idx = np.random.choice(len(pts), 2, replace=False)      # 2-point sample
        p1, p2 = pts[idx]
        if np.allclose(p1, p2): continue
        dx, dy = p2 - p1
        norm = np.hypot(dx, dy)
        a, b = -dy/norm, dx/norm                                # unit normal
        d = -(a*p1[0] + b*p1[1])
        # perpendicular distance |ax + by + d| < thresh
        inliers = np.where(np.abs(a*pts[:,0] + b*pts[:,1] + d) < thresh)[0]
        if len(inliers) > len(best_inliers):
            best_inliers, best_sample, best_params = inliers, idx, (a, b, d)
    return best_params, best_sample, best_inliers

line_params, line_sample_idx, line_inlier_idx = ransac_line(X, max_iters=2000, thresh=1.2)
remnant_mask = np.ones(len(X), dtype=bool)
remnant_mask[line_inlier_idx] = False                           # remove line inliers
remnant_indices = np.where(remnant_mask)[0]
X_remnant = X[remnant_mask]

def circle_loss(params, pts):
    cx, cy, rad = params
    return np.sum((np.hypot(pts[:,0]-cx, pts[:,1]-cy) - rad)**2)

def ransac_circle(pts, max_iters=2000, thresh=1.0):
    best_inliers, best_sample, best_circle = [], None, None
    for _ in range(max_iters):
        idx = np.random.choice(len(pts), 3, replace=False)      # 3-point sample
        circle = fit_circle_3pts(*pts[idx])
        if circle is None: continue
        cx, cy, rad = circle
        inliers = np.where(np.abs(np.hypot(pts[:,0]-cx, pts[:,1]-cy) - rad) < thresh)[0]
        if len(inliers) > len(best_inliers):
            best_inliers, best_sample, best_circle = inliers, idx, circle
    refined = minimize(circle_loss, x0=best_circle, args=(pts[best_inliers],),
                       method='Nelder-Mead')                    # refine using all inliers
    return refined.x, best_sample, best_inliers

circle_params, circle_sample_idx, circle_inlier_remnant_idx = ransac_circle(
    X_remnant, max_iters=2000, thresh=1.0)
```

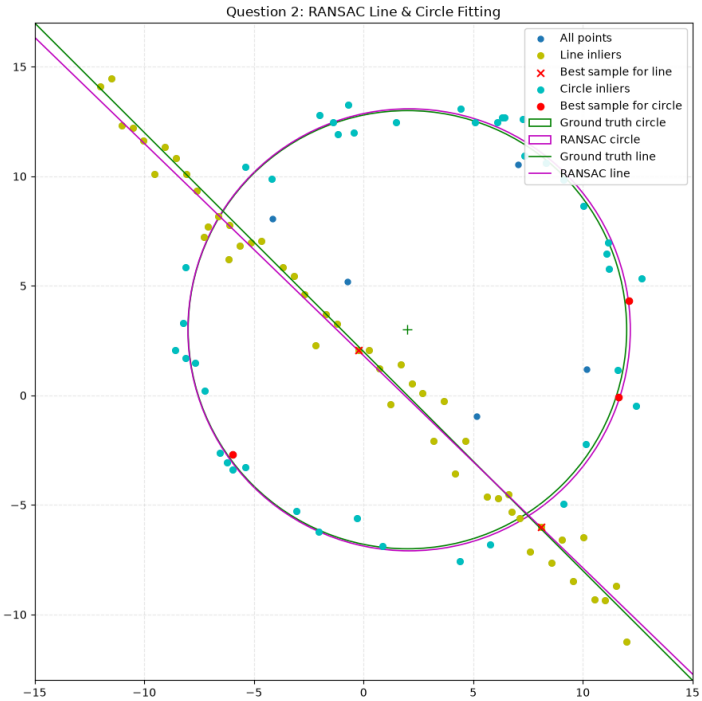


Figure 2: RANSAC line and circle (magenta) closely match the ground truth (green).

(d) What happens if we fit the circle first?

If the circle is fitted first, line points near the two intersections fall inside the circle's inlier band and are removed with it, so fewer points are left for the line fit. Fitting the line first (2-point sample, which cannot imitate a circle) is therefore more reliable.

3. Question 3: Superimposing Images Using a Homography

```
# points: 4 corners clicked on the background (TL, TR, BR, BL) using cv.setMouseCallback
dst_pts = np.float32(points)
h, w = overlay_bgr.shape[:2]
src_pts = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]])

H = cv.getPerspectiveTransform(src_pts, dst_pts)
warped = cv.warpPerspective(overlay_bgr, H, (background_bgr.shape[1], background_bgr.shape[0]))
mask_src = np.full((h, w), 255, dtype=np.uint8)
mask = cv.warpPerspective(mask_src, H, (background_bgr.shape[1], background_bgr.shape[0])) > 0

alpha = 0.9                                                    # 90% overlay, 10% background
result = background_bgr.copy()
result[mask] = cv.addWeighted(background_bgr, 1 - alpha, warped, alpha, 0)[mask]
```



Figure 3: Images superimposed using a homography ($\alpha = 0.9$). Left to right: Wadham College and the Union Jack; a locomotive and a Sri Lanka tourism image; a Times Square billboard and a peacock image.

Rationale for the image selection

A homography maps one plane onto another, so each background was chosen to contain a clearly bounded flat surface: the Wadham College wall (given), the side of a Srilankan locomotive and a billboard in Times Square. The two extra pairs follow a "Visit Sri Lanka" theme and show realistic uses, a train livery wrap and virtual billboard advertising, under different amounts of perspective distortion.

4. Question 4: PCB Image Alignment and Difference Detection

```
sift = cv.SIFT_create()
kp_ref, des_ref = sift.detectAndCompute(reference_gray, None)   # reference: AECC1.jpg
matcher = cv.BFMatcher()

for path in image_paths:
    image = cv.imread(path)
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    kp_img, des_img = sift.detectAndCompute(gray, None)

    matches = matcher.knnMatch(des_img, des_ref, k=2)
    good = [m for pair in matches if len(pair) == 2 for m, n in [pair]
            if m.distance < 0.75 * n.distance]                    # Lowe's ratio test

    src_pts = np.float32([kp_img[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp_ref[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, inliers = cv.findHomography(src_pts, dst_pts, cv.RANSAC, 5.0)

    aligned = cv.warpPerspective(image, H, (reference.shape[1], reference.shape[0]))
    difference_images.append((os.path.basename(path), cv.absdiff(reference, aligned)))
```

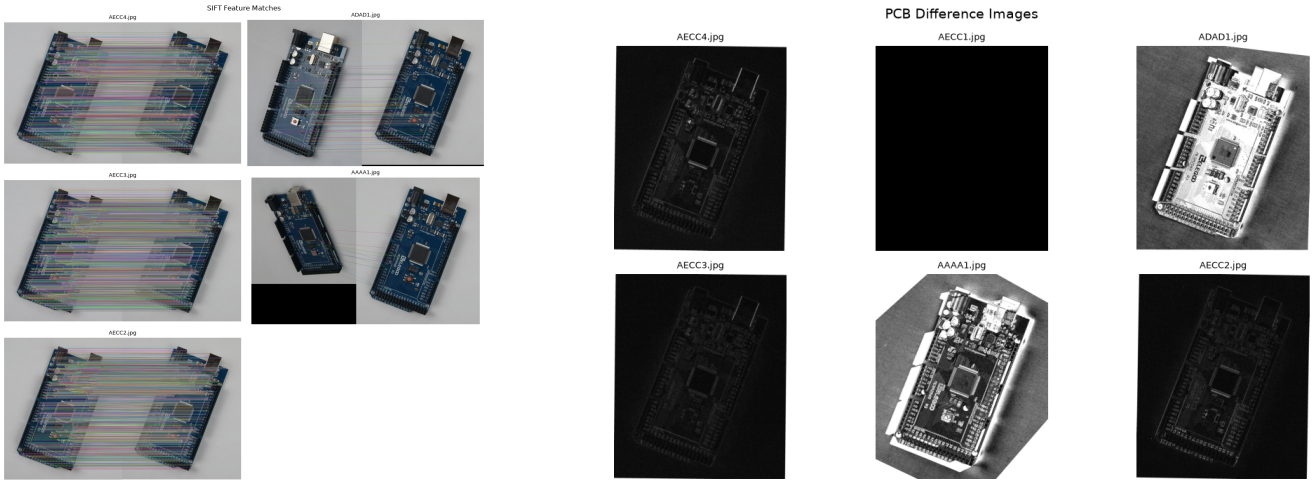


Figure 4: Left: SIFT matches (RANSAC inliers) with the reference `AECC1.jpg`. Right: difference images after alignment (display range 0–50); AECC1 (reference vs itself) is fully black and AECC2–AECC4 are almost black, while ADAD1 and AAAA1 show large differences.

(e) Reason for high differences

The homography aligns only the plane of the PCB, so tall components such as the USB Type-B port, capacitors and reset button shift with any change in viewpoint (parallax), and their shiny metal surfaces and shadows change with the lighting. Differences in component position, shape or package between boards, and small residual alignment errors at sharp edges, also give high values.